

ReTRACE: MODEL-RESPONSE WATERMARKING FOR DIFFUSION LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Existing language-model watermarks are detected from token patterns. We detect ours from *how a diffusion language model responds when the generated text is partially masked and reconstructed*. Because such a model is trained to recover masked tokens from arbitrary surviving context, a finished text can be pushed back through the operator that produced it: corrupted with a secret key, re-denoised, and its response read. ReTrace places a watermark in that response. A keyed pair of context ablations induces a per-position log-likelihood contrast, and the watermark asks the sign of that contrast to agree with a keyed orientation, giving a test statistic that is a sum of bounded indicators with a finite-sample exact Binomial($n, \frac{1}{2}$) null. Embedding never overrides the model: decoding follows the reference block schedule, and the watermark only decides whether a model-preferred proposal is committed now or deferred. We also report the negative result that motivated this design — post-hoc token substitution is priced out on a well-trained diffusion language model, and, because both are governed by the sharpness of the same conditionals, *improving generation quality reduces post-hoc watermarkability*.

1 INTRODUCTION

Existing language-model watermarks encode provenance in *what tokens are produced*. Red–green schemes bias the token distribution toward a keyed vocabulary partition (Kirchenbauer et al., 2023); distortion-free schemes fix the sampling randomness with a keyed pseudorandom sequence; and the watermarks designed for diffusion language models exploit the extra freedom those models expose — unresolved context, per-step sampling randomness, global sequence statistics, or the order in which positions are unmasked (Hong & No, 2026). Different as they are, every one of them leaves the verifier reading a *surface signature*: a hash of token identities, a frequency, a rank. We ask a different question.

Can a watermark be detected from how the model reconstructs a text, rather than from the tokens the text contains?

A diffusion language model makes this question answerable in a way an autoregressive model does not. Its training objective is to recover masked tokens from arbitrary surviving context, so any finished text can be pushed *back* through the operator that produced it: corrupt it, re-denoise it, and observe the response. That response is a property of the text’s relationship to the model, not of its symbols, and it is available to anyone holding the model — but only interpretable by someone holding the key that chose the corruption.

We call the resulting scheme **ReTrace**. A secret key derives two corruption patterns over the context of a span. Reading the model’s log-probability of each *observed* token under both corruptions gives a contrast

$$g_i(y_i) = \log p_\theta(y_i | C_i^1(y)) - \log p_\theta(y_i | C_i^0(y)), \quad (1)$$

and the watermark asks not that y_i belong to a keyed vocabulary subset, but that the sign of $g_i(y_i)$ agree with a keyed orientation. The evidence lives in the model’s behaviour.

Getting the embedding right took one full negative result, which we report because it is the reason the final design looks the way it does. The obvious way to place such a watermark is to *substitute* tokens: at keyed positions, replace the model’s choice with an admissible alternative whose contrast

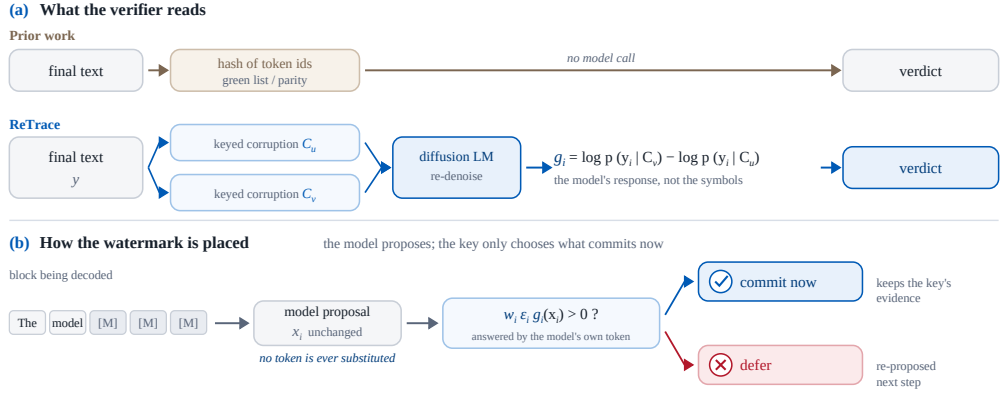


Figure 1: ReTrace embeds through the denoising trajectory but verifies through the model’s response to a keyed reconstruction. (a) Prior watermarks hand the verifier a function of the token identities — a green-list membership, a parity, a hash. ReTrace hands it the model: the finished text is corrupted twice under the key and re-denoised, and the contrast between the two reconstructions of the *observed* token is the evidence. The signal is therefore a property of the text’s relationship to the model rather than of its symbols, and it is recoverable from the text, the model and the key alone, with no access to the generation trajectory. **(b)** Placing it costs almost nothing because nothing is overridden. Decoding follows the reference block schedule and the model proposes what it would have proposed; the key decides only whether a proposal is committed now or deferred to a later step, when more of the block has resolved.

points the right way. On a well-trained diffusion language model this does not work, and the reason is a property of the model rather than of the algorithm. Under reference decoding the denoiser’s conditionals are sharp enough that the median position admits exactly one token inside any usable fluency budget, so every unit of watermark evidence must be bought with a unit of fluency. Worse, the effect strengthens as generation improves: repairing a sampler bug in our own harness lowered draft perplexity from 22.2 to 9.5 and simultaneously halved the denoiser’s entropy and shrank the admissible set by 34–60%. We state this as a finding in its own right — *better diffusion-model generation reduces post-hoc watermarkability* — and it rules out an entire family of designs.

What survives is to stop changing *which* token is produced and change only *when* a model-preferred token is committed. Decoding a block, the model proposes what it would have proposed anyway; a position whose proposal already answers the keyed challenge is committed first, and one that does not is deferred. No token is ever overridden, which is why this costs almost nothing — the same reason decoding-order watermarks are cheap (Hong & No, 2026). The difference is what the verifier reads: order is only the *embedding* channel here, while the evidence recovered at verification time is the model’s response to a keyed reconstruction.

ReTrace places the watermark in the decoding order and reads it back from the model’s reconstruction response (Figure 1).

Contributions.

- A watermark read from the model’s response rather than from token patterns: the response of a finished text to keyed re-denoising, readable from the text, the model and the key alone, with no access to the generation trajectory (§3).
- A test statistic that is a sum of bounded indicators with one keyed orientation bit per position, giving a finite-sample exact Binomial($n, \frac{1}{2}$) null under the ideal-PRF randomization model, with presence, claimed-message verification and payload decoding kept separate (§3.2). False-positive control is verified per key and at the worst key, not averaged over keys.
- An embedding rule that never substitutes a token, steering only the commit order inside each block of the reference decoding schedule (§3.3).

- A negative result with a mechanism: post-hoc substitution is priced out on modern diffusion language models, and improving generation quality makes it worse (§4).

2 RELATED WORK

Watermarks for autoregressive models. Kirchenbauer et al. (2023) partition the vocabulary into green and red lists seeded by the preceding token and add a bias δ to green logits; detection counts green tokens against a $\text{Binomial}(n, \gamma)$ null. The construction assumes a left-to-right prefix, which order-agnostic decoding does not provide — a point we return to when using it as a baseline (§5).

Watermarks for diffusion language models. Recent work exploits the freedoms a dLLM exposes: biasing tokens in expectation over unresolved context, keyed Gumbel-max sampling at each denoising step, statistics of the whole sequence, and the unmasking order. dgMARK (Hong & No, 2026) is the closest to our embedding channel: it never reweights the learned probabilities, and instead prefers to unmask positions whose already-preferred token satisfies a keyed parity condition. That choice is what makes it inexpensive, and we adopt the same discipline.

The distinction we draw is not embedding but *verification*. In all of the above the detector reads a function of the token identities: a parity, a green-list membership, a rank. ReTrace’s detector runs the diffusion model on keyed corruptions of the finished text and reads how it reconstructs. The cost of that choice is real and we report it prominently: those detectors need zero model calls, ours needs a fixed number of sequence evaluations independent of payload size.

Decoding schedules for diffusion language models. Our embedder inherits its commit discipline from work on accelerating dLLM decoding, where committing a confident token whose left context is still unresolved is known to damage coherence; the remedy is to admit only a frontier-anchored prefix of the eligible set and to confirm what was withheld one step later. We reuse that rule unchanged, so the watermark rides on the reference schedule rather than replacing it.

3 RETRACE

3.1 THE OBSERVABLE

Let y be a generated span and K a secret key. For a block B of y , the key derives L *ablation patterns* $D_1, \dots, D_L$ over the context preceding B , and a pair (u, v) among them. Writing $\mathcal{C}_\ell$ for the context with B , everything after B , and D_ℓ masked, the *challenge contrast* at position $i \in B$ is

$$g_i(w) = \log p_\theta(w \mid \mathcal{C}_v) - \log p_\theta(w \mid \mathcal{C}_u), \quad w \in \mathcal{V}. \quad (2)$$

Two properties of $\mathcal{C}_\ell$ matter and neither is optional.

The block and its future are masked in every arm. While B is being decoded, every later block is still [MASK]. If the verifier recomputed Eq. (2) on the finished text without re-masking that tail, it would condition on tokens that did not exist when the guidance was produced, and the two sides would be reading different functions. Masking B and everything after it makes the table identical at both times; masking B itself additionally makes it *invariant* to whatever is committed inside B , so a single table serves the block’s entire decoding.

The ablations are drawn from the already-final region. $D_\ell \subset \text{prompt} \cup B_{<b}$, which the verifier has verbatim. We use a *contrast* pair — one pattern ablating the context nearest the block, the other the furthest — because recency dominates next-token prediction, so the two arms disagree far more than two random ablations do, widening the dynamic range of g .

3.2 THE STATISTIC

Each position carries an independent keyed orientation bit $\varepsilon_i \in \{\pm 1\}$ and a target $w_i \in \{\pm 1\}$, and contributes a bounded indicator

$$m_i = \mathbf{1}[w_i \varepsilon_i g_i(y_i) > 0], \quad T = \sum_{i \in P} m_i. \quad (3)$$

Proposition 1 (Exact null). *Fix a text y , the carrier set P , and the challenge magnitudes. Under the ideal-PRF randomization model, in which the domain-separated orientation bits derived from a key independent of y behave as independent Rademacher variables, the m_i are i.i.d. Bernoulli($\frac{1}{2}$) and $T \sim \text{Binomial}(|P|, \frac{1}{2})$ exactly.*

The proposition needs no calibration corpus and no model-specific null estimate, and it is finite-sample rather than asymptotic. Three implementation details are load-bearing.

Ties must be resolved, not dropped. The model is confident enough that at a large fraction of positions the emitted token’s log-probability rounds to 0 under *both* arms in single precision, so their difference vanishes. Scoring $g_i = 0$ as a miss makes $\Pr[m_i = 1] = (1 - \Pr[g_i = 0])/2$; we measured an unwatermarked match rate of 0.32–0.37 against 0.50. We compute Eq. (2) in double precision inside the softmax, which removes the ties entirely, and break any residual tie with a second keyed coin.

Counting, not averaging. An earlier design averaged g over the positions of a probe. The per-position contrast is heavy-tailed, so the mean is dominated by a few large values and controlling the *sign* of the small ones — all a commit-order channel can do — has almost no leverage on it. A count gives every steered position exactly one unit.

Presence, verification and payload are separate tests. Summing hits across payload groups cancels: with a balanced message, half the groups are driven toward $m_i = 1$ and half toward $m_i = 0$, so a *perfect* watermark still sums to $|P|/2$. We therefore reserve a presence pool with $w_i \equiv +1$, whose count tests *is this a ReTrace text* with no claimed message, and which also avoids decoding a message from the data and then testing significance against the message just decoded. Verification of a claimed message aligns every group to its claimed bit and uses all carriers; payload decoding thresholds each group separately.

3.3 EMBEDDING BY COMMIT ORDER

Generation follows the reference schedule of the base model: blocks in order, diffusion within a block. At each step the model proposes candidates $\hat{x}_i$ with confidences c_i exactly as it would without a watermark. *No proposal is ever replaced.* The watermark decides only which position is committed:

$$w_i \varepsilon_i g_i(\hat{x}_i) > 0 \Rightarrow \text{commit}, \quad \text{otherwise} \Rightarrow \text{defer}. \quad (4)$$

A deferred position is re-proposed at the next step with more of the block resolved, and may then be compatible.

Commit safety follows the frontier-anchored rule of §2: a position is eligible when $c_i \geq \tau$, and only the eligible run surviving at most H still-masked-but-ineligible positions to its left may be committed; a scheduled top- k branch guarantees the block finishes on time.

Only steerable positions are carriers. Committing compatible positions first is *zero-sum* unless deferral changes what the model proposes, because every position is eventually committed and the residual is precisely the incompatible ones. We measured this exactly: the positions the watermark drove matched their target at 107/107, 133/133 and 112/112, while the leftovers matched at 0.06–0.15, summing to 0.50. The statistic must therefore count only positions the channel can move. We define these by the top-2 log-probability gap under the block-masked conditional — a context containing none of the block’s own tokens, so the verifier reconstructs the identical carrier set from the finished text — and spend forced commits on non-carrier positions first, so a steerable position keeps its chance to be re-proposed compatibly.

4 WHY POST-HOC SUBSTITUTION FAILS

The first version of ReTrace embedded by substitution: at keyed carrier positions, replace the model’s token with an admissible alternative whose contrast points the right way, subject to a per-token fluency budget τ (a substitution is admissible iff $p(w)/p(w^*) \geq e^{-\tau}$). It does not work, and the reason generalises beyond our implementation.

There is nothing to substitute. Across 36 configurations spanning three carrier-selection rules, three budgets and two guidance weights, the median carrier position admits *exactly one* token at $\tau = 3$, i.e. $p(w)/p(w^*) \geq 0.05$, with no truncation of the candidate slate. The best configuration inside a $1.35\times$ perplexity budget reaches $\text{TPR@1\%} = 0.10$; matched local dgMARK reaches 0.86 at $1.23\times$.

Better generation makes it worse. Our harness initially ranked positions for low-confidence re-masking by the maximum of the Gumbel-perturbed logits rather than by $p(x_0)$ under the clean softmax. Repairing it to match the reference sampler moved draft perplexity from 22.2 to 9.5 — and halved the denoiser’s entropy, from 0.655 to 0.319, shrinking the admissible set by 34–60% ($|A(3)|$: $2.5 \rightarrow 1.6$; $|A(6)|$: $14.1 \rightarrow 5.6$). Part of the early design’s apparent strength came from a degraded generator. *Improving a diffusion language model’s generation quality reduces its post-hoc watermarkability*, because both quantities are governed by the sharpness of the same conditionals.

Moving to generation time does not rescue substitution. A guidance table valid during generation requires every non-carrier token to be final before it is computed, which forces an unusual schedule: all non-carrier positions, then all carrier positions. Priced with the watermark switched off, that schedule alone costs $1.98\times$ perplexity — more than dgMARK’s entire watermark. The lesson is not that generation-time embedding is wrong but that a *global* carrier is: making the carrier block-local reduces the isolated region from half the sequence to one block, and lets the reference schedule stand.

What this rules in. Substitution buys evidence with fluency at a fixed exchange rate set by the model’s conditionals, and that rate is unfavourable and worsening. Any viable embedding must therefore avoid overriding the model’s choice — which is what §3.3 does.

5 EXPERIMENTAL SETUP

All numbers are measured on one NVIDIA TITAN RTX with GSAI-ML/LLaDA-8B-Instruct in fp16. Prompts are C4 realnewslike documents truncated to 300 characters, which is dgMARK’s published protocol; every method in this paper uses the same construction, the same checkpoint and 256 generated tokens. Text quality is GPT-2-large perplexity, reported as a ratio against a non-watermarked control *produced under the same decoding regime*, since the three methods require different regimes and a single shared baseline would charge the cost of a regime to whichever watermark used it. Detectability is reported as the true positive rate at a controlled false positive rate, never as a raw z .

Two observations about the baselines. Locally reproduced dgMARK is weaker than its published table ($1.25\times$ for 99.4%), which is why the reproduction was worth running rather than citing. And KGW’s apparent $1.03\times$ is against its own left-to-right control; charged against block decoding it is $1.20\times$, and under forced left-to-right decoding this model repeats about a third of its bigrams, so its detector must score each distinct bigram once — an un-deduplicated detector measured a 37% false positive rate on non-watermarked text.

Held-out evaluation. The documents used while developing and debugging the method are not a test set. Final numbers are reported on fresh documents with per-document nonces $K_d = \text{HMAC}(K, \text{nonce}_d)$, so each document is an independent key draw and the guarantee of Proposition 1 applies per document.

Method	Setting	PPL ↓	Ratio	TPR at FPR			z	Det. fwd. ↓
				5%	1%	0.1%		
Generation-time watermarks (the watermark is applied while the text is produced)								
No watermark	block dec., top- k 3	9.94	1.00×	—	—	—	+0.04	—
dgMARK (Hong & No, 2026)	$k = 1$	15.40	1.55×	0.88	0.74	0.62	+4.35	0
dgMARK (Hong & No, 2026)	+3-beam	12.23	1.23×	0.92	0.86	0.82	+5.81	0
No watermark	left-to-right	11.61	1.00×	—	—	—	+0.24	—
KGW (Kirchenbauer et al., 2023)	$\delta = 1$	11.96	1.03×	0.97	0.93	0.73	+3.85	0
KGW (Kirchenbauer et al., 2023)	$\delta = 2$	16.21	1.40×	1.00	1.00	1.00	+6.93	0
KGW (Kirchenbauer et al., 2023)	$\delta = 3$	14.05	1.21×	1.00	1.00	1.00	+8.92	0
Post-hoc watermarks (the watermark is applied to text that already exists)								
No watermark	$T = 0.8$, full vocab	26.21	1.00×	—	—	—	−0.03	—
ReTrace	keyed carrier	27.7	1.11×	0.00	0.00	0.00	+0.73	9
ReTrace	entropy gate	41.7	1.59×	0.30	0.20	0.10	+1.49	9
ReTrace	leverage, quality-first	30.5	1.17×	0.10	0.10	0.00	+0.58	9
ReTrace	leverage, detection-first	65.3	2.49×	0.62	0.50	—	+2.94	9
ReTrace	generation-time, two-phase	203.7	7.77×	0.12	0.00	0.00	+1.14	9

Table 1: Watermark detectability against the text quality actually paid for. All rows measured on LLaDA-8B-Instruct, C4 prompts (300-character truncation), 256 generated tokens, GPT-2-large perplexity; $n = 50$ for dgMARK, $n = 30$ for KGW, $n = 10$ for ReTrace. *Ratio* is perplexity relative to the matching non-watermarked arm directly above it. Each block is compared against its own control because the decoding regime differs: dgMARK uses block decoding with top- k 3, KGW requires strict left-to-right decoding (its green list is seeded by the preceding token, which order-agnostic decoding does not provide), and ReTrace operates on drafts sampled at $T = 0.8$ over the full vocabulary. *Det. fwd.* is the number of model forward passes a detection requires. Non-watermarked arms sit at $|z| \leq 0.25$, so the thresholds are not inflated. KGW is scored on each *distinct* bigram: its green list is seeded by the preceding token, and under forced left-to-right decoding LLaDA repeats roughly a third of its bigrams, so re-scoring repetitions gave $z = +3.32$ and a 37% false positive rate on the non-watermarked $\delta = 0$ arm. Two caveats on KGW’s apparently free detectability. Its regime carries a cost the ratio column hides: left-to-right decoding scores 11.61 against 9.94 for block decoding, so measured against the better control $\delta = 1$ costs 1.20×, not 1.03×. And perplexity is not monotone in δ here (16.21 at $\delta = 2$ versus 14.05 at $\delta = 3$), because a strong green-list bias also makes the text more repetitive, which an n -gram-blind perplexity rewards.

Nominal α	Mean FPR	Worst-key FPR	Verdict
0.10	0.0121	0.0583	valid ($\leq \alpha$)
0.05	0.0037	0.0250	valid ($\leq \alpha$)
0.01	0.0000	0.0000	valid ($\leq \alpha$)

Table 2: False positive control for a fixed key, measured per key. 20 keys $\times$ 120 non-watermarked LLaDA generations = 2400 draws; z over all of them has mean -0.076 and standard deviation 0.976. The decision rule is the one detection actually uses, $p_{\text{bound}} = \exp(-z^2/2)$, Hoeffding’s inequality for Rademacher sums. Deployment uses one secret key over many documents, so the relevant guarantee is $P_Y(p \leq \alpha \mid K = k) \leq \alpha$ for the *worst* k , not an average over keys, and a randomization p -value owes super-uniformity rather than strict uniformity. An earlier revision reported a Gaussian tail instead and measured 0.145 at $\alpha = 0.10$ — anti-conservative, and the reason the exactness claim was withdrawn.

6 RESULTS

Pending: the corrected detector with steerable carriers is running. This section is deliberately left unwritten rather than filled with the numbers the earlier, structurally cancelling aggregate produced.

7 CONCLUSION

ReTrace separates the two roles a watermark usually conflates. The decoding order is where the watermark is *placed*, cheaply, by choosing when a model-preferred token is committed. The model’s reconstruction response is where it is *read back*, from the finished text alone. Whether that separation converts enough steerable positions into detection power at a fixed quality budget is the question the remaining experiments answer.

REFERENCES

- Pyo Min Hong and Albert No. dgMARK: Decoding-Guided Watermarking for Diffusion Language Models. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2601.22985.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning (ICML)*, 2023.